

Компютърни архитектури

Ивайло Андреев + gemini 3 Flash

31 май 2026

Съдържание

1	Обща структура на компютрите и концептуално изпълнение на инструкциите, запомнена програма	2
1.1	Обща структура на компютрите	2
1.1.1	Организация на комуникацията: Интерфейсна шина	2
1.2	Концептуално изпълнение на инструкциите	3
1.2.1	Оптимизационни хардуерни стратегии	3
1.3	Запомнена програма	4
2	Формати на данните	4
2.1	Цели двоични числа	4
2.2	Двоично-десетични числа (BCD)	5
2.3	Двоични числа с плаваща запетая	6
2.4	Символни данни и кодови таблици	6
3	Вътрешна структура на централен процесор	6
3.1	Регистри	7
3.2	Аритметико-логическо устройство (АЛУ)	7
3.3	Регистър на състоянието и флагове (Flags Register)	7
3.4	Блок за управление (Управляващо устройство - УУ)	8
4	Инструкции на централен процесор	8
4.1	Структура на машинната инструкция	8
4.2	Модели на адресация на операндите	8
4.3	Функционални групи инструкции	9
4.3.1	Аритметико-логически инструкции	9
4.3.2	Низови инструкции (String Instructions)	9
4.3.3	Безусловни и условни преходи	9
4.3.4	Управление на програмата	10

1 Обща структура на компютрите и концептуално изпълнение на инструкциите, запомнена програма

1.1 Обща структура на компютрите

Персоналният компютър (ПК) представлява универсална цифрова изчислителна система, предназначена за решаване на разнообразни алгоритмични задачи и проблеми. Основните му специфични особености включват компактни размери, ограничена и проста система от команди, ограничен обем на основната памет и опростен интерфейс за свързване на компонентите.

Въпреки голямото разнообразие от производители, персоналните компютри се характеризират с еднакъв модел на вътрешната архитектура, базиран на класическия модел на Фон Нойман, който се състои от три основни хардуерни компонента:

- **Централен процесор (ЦП / CPU):** Представлява основната изчислителна сила на компютъра, която изпълнява всички аритметични и логически операции и управлява жизнения цикъл на инструкциите.
- **Оперативна памет (ОП / RAM):** Енергозависима памет, която позволява бързо четене и писане. В нея се съхраняват изпълняваните в момента програми и файлове с данни, с които ЦП работи директно. При архитектурата на Фон Нойман както данните, така и програмите се съхраняват в едно и също споделено пространство на ОП.
- **Входно-изходни (периферни) устройства (ВИ):** Служат за предаване или получаване на информация от/към външната среда (клавиатура, мишка, монитор, мрежова карта NIC и др.).

1.1.1 Организация на комуникацията: Интерфейсна шина

Обменът на сигнали, команди и данни между компонентите може да се организира по два начина:

1. **Индивидуални информационни канали (връзка "всеки с всеки"):** Компонентите образуват пълен граф на свързаност. Скоростта е висока, но сложността и цената нарастват неимоверно с добавянето на нови устройства.
2. **Обща информационна шина:** Споделена комуникационна магистрала, достъпна за всички устройства в конкурентен режим. Тя е по-евтина и лесна за надграждане, но в даден момент могат да комуникират само две устройства, което може да се превърне в теснолинейка на производителността.

Поради конкурентния достъп се въвежда специализирано хардуерно устройство – **арбитър (контролер) на шината**, който управлява приоритетите и определя кое устройство кога да заеме линията. На ЦП се дава най-висок приоритет. Логически шината се разделя на три подшини според функциите си:

- **Подшина за данни:** Пренася информационните битове между ЦП, паметта и ВИ. Тя е двупосочна. Ширината ѝ определя броя битове, пренасяни едновременно.
- **Подшина за адреси:** Еднопосочна подшина, определяща конкретното физическо местоположение в паметта или ВИ. Нейната ширина определя максималното количество адресируема памет (напр. 16-битова шина адресира $2^{16} = 64$ KB, а 32-битова – 4 GB).
- **Управляваща подшина:** Пренася електрически контролни сигнали (за четене/писане, линии за синхронизация, заявки за прекъсвания и сигнали за статус).

1.2 Концептуално изпълнение на инструкциите

Изпълнението на всяка машинна инструкция в процесора преминава последователно през следните пет базови етапа:

1. **Извличане (Fetch):** Инструкцията се прочита от ОП от адреса, указан в програмния брояч (*IP/PC*).
2. **Дешифриране (Decode):** Кодът на операцията се анализира от управляващия блок и понякога се разбива на елементарни микрооперации (μ -ops).
3. **Осигуряване на операнди (Fetch Operands):** Изчисляват се адресите на входните данни и те се зареждат от паметта или регистрите.
4. **Изпълнение (Execute):** Аритметико-логическото устройство извършва съответната операция.
5. **Запис на резултата (Writeback):** Получената стойност се записва в целевия регистър или в ОП.

1.2.1 Оптимизационни хардуерни стратегии

Съвременните суперскаларни процесори използват паралелизъм на ниво инструкции за ускоряване на работата:

- **Конвейерна обработка (Pipelining):** Припокриване на етапите на изпълнение. Докато една инструкция се изпълнява, следващата се дешифрира, а последващата се извлича. Това предотвратява престоя на хардуерните блокове.
- **Изпълнение извън реда (Out-of-Order Execution):** Инструкциите се пренареждат динамично така, че тези, които имат готови операнди, да се изпълнят веднага, без да чакат забавени предходни инструкции.

- **Предсказване на преходи (Branch Prediction):** Статистическо отгатване на посоката на условните преходи с цел конвейерът да не спира работа при чакане на оценка на флаговете.
- **Спекулативно изпълнение (Speculative Execution):** Изпълнение на инструкции по предположения път преди преходът да е реално изчислен. При грешка резултатите се отхвърлят.
- **Преименуване на регистри (Register Renaming):** Използване на допълнителни скрити физически регистри за елиминиране на фалшиви зависимости между данните (архитектурни конфликти).

1.3 Запомнена програма

Концепцията за запомнена програма изисква кодът да се съхранява в паметта като поредица от числа. При **архитектурата на Фон Нойман** програмите се зареждат от постоянни носители (ROM, HDD/SSD) в общата ОП преди изпълнение. При **Харвард архитектурата** съществуват две физически разделени памети – една изрично за инструкции и една за данни.

В съвременните системи всяка програма съдържа една или повече **нишки (threads)**, които се изпълняват конкурентно и могат да бъдат прекъсвани (от ОС или хардуерни събития). За да бъде това прекъсване напълно "прозрачно" за програмата, хардуерът и ОС съхраняват т.нар. **векторно състояние на процеса** (съдържание на програмния брояч, стековия указател, регистрите с данни и управляващите таблици). След обработка на прекъсването векторът се възстановява и процесът продължава без загуба на контекст. Важно правило в софтуерното инженерство е, че изпълнимият код не трябва да се видоизменя динамично по време на работа (самомодифициращ се код се счита за лош стил).

2 Формати на данните

В паметта на компютъра всички данни се представят в двоичен вид (като битове 0 и 1). Чрез n -битово поле могат да се кодират точно 2^n различни състояния или стойности.

2.1 Цели двоични числа

Целите числа се представят в машинни думи или байтове с фиксирана дължина (8, 16, 32, 64 бита). Те биват два вида:

- **Без знак (Unsigned):** Всички n бита се използват за представяне на абсолютната стойност. Диапазонът е от 0 до $2^n - 1$.
- **Със знак (Signed):** Най-старшият бит (a_{n-1}) се отделя за знак (0 за положително, 1 за отрицателно число).

За кодиране на отрицателни числа исторически съществуват три основни метода:

1. **Прав код (Sign-Magnitude):** Старшият бит е знак, останалите са модулът на числото. Диапазонът е $[-2^{n-1} + 1, 2^{n-1} - 1]$. Недостатък: Нулата има две кодирания (+0 и -0), а аритметичните операции се реализират много трудно апаратно.
2. **Обратен код (Ones' Complement):** Отрицателните числа се образуват чрез инвертиране (обръщане на 0 в 1 и обратно) на правия код. Нулата отново има две представяния, а при пренос извън най-старшия бит при събиране, към крайния резултат трябва циклично да се добави 1.
3. **Допълнителен код / Допълнение към двойката (Two's Complement):** Най-широко използваната схема в съвременните процесори. Отрицателно число се получава чрез инвертиране на битовете на положителното и добавяне на единици (+1) към най-младшия бит.

Математически, стойността на едно N -битово число в допълнителен код се изчислява по следната формула:

$$\overline{a_{N-1}a_{N-2}\dots a_{0_{2c}}} = -a_{N-1} \cdot 2^{N-1} + \sum_{i=0}^{N-2} a_i \cdot 2^i$$

Следователно диапазонът е асиметричен: $[-2^{N-1}, 2^{N-1} - 1]$. Нулата има само едно единствено уникално представяне ($\overline{00\dots 0_{2c}}$). Предимството на допълнителния код е, че хардуерът извършва събиране и изваждане по един и същ начин, а преносът извън най-старшия бит просто се игнорира, което улеснява логическите схеми.

2.2 Двоично-десетични числа (BCD)

При BCD формата (*Binary-Coded Decimal*) всяка десетична цифра от 0 до 9 се представя самостоятелно чрез 4-битово двоично поле (тетрада). Тъй като с 4 бита могат да се представят 16 комбинации, 6 от тях остават неизползвани (невалидни). Съществуват две форми за организация в байтове:

- **Непакетиран BCD формат:** В един байт се съхранява само една десетична цифра (обикновено в по-младата тетрада, а старшата се запълва с нули).
- **Пакетиран BCD формат:** В един байт се съхраняват две десетични цифри (една в старшата и една в младшата тетрада).

Предимство: Липсват хардуерни загуби от закръгляне при конвертиране на десетични дробни числа в двоични (напр. числото 0.3 е безкрайна периодична двоична дроб, но в BCD се кодира точно). **Недостатък:** Форматът е силно неефективен по отношение на паметта и изчисленията, изисква допълнителни коригиращи инструкции след аритметични операции.

2.3 Двоични числа с плаваща запетая

Рационалните реални числа се представят в компютъра чрез експоненциален запис с плаваща запетая (*Floating-Point*). Те апроксимират реалните числа и се състоят от три компонента: знак на числото (s), мантиса (m) и експонента (e). Стойността се определя като:

$$\text{Value} = (-1)^s \times m \times 2^e$$

Съвременните архитектури имплементират международния стандарт **IEEE 754**, който дефинира следните основни формати:

- **binary32 (Single Precision / float)**: Заема общо 32 бита – 1 бит за знак, 8 бита за експонента и 23 бита за мантиса.
- **binary64 (Double Precision / double)**: Заема общо 64 бита – 1 бит за знак, 11 бита за експонента и 52 бита за мантиса.

Мантисата е нормализирана (във формат $1.f$), като водещата единица пред десетичната запетая не се пази физически в паметта (скрит бит), което добавя допълнителна точност. Експонентата се съхранява с отместване (*bias*), за да се избегне поддържането на знак в нея.

2.4 Символни данни и кодови таблици

Символите и текстовете се представят като поредици от числа посредством стандартизирани **кодови таблици**:

- **ASCII**: 7-битов (или разширен 8-битов) стандарт. Кодовете от 0 до 127 са твърдо дефинирани за латиницата, цифрите и контролните символи. Регионът [128, 255] зависи от локалната кодова страница (напр. CP1251 за кирилица), което често води до проблеми с нечетими текстове при грешна интерпретация.
- **Unicode (UTF-8, UTF-16, UTF-32)**: Универсална система, обхващаща всички писмености. **UTF-8** е с променлива дължина (от 1 до 4 байта) и е напълно обратно съвместим с ASCII за първите 127 символа. **UTF-16** и **UTF-32** използват фиксирани или по-големи базови структури от 2 или 4 байта.

3 Вътрешна структура на централен процесор

Централният процесор съдържа вътрешни подблокове, които си взаимодействат по вътрешнопроцесорна шина:

3.1 Регистри

Регистрите представляват свръхбързи паметови клетки, разположени директно в ядрото на процесора. Техният размер варира от 8 до 512 бита, като стандартно съвпадат с разрядността на архитектурата (машинната дума). Класифицират се като:

- **Регистри с общо предназначение (GPR):** Използват се от програмиста за съхранение на междинни данни при операции (в x86 това са EAX, EBX, ECX, EDX и техните 64-битови аналози RAX, RBX...).
- **Програмен брояч / Указател на инструкции (IP/PC):** Пази адреса на следващата инструкция, която трябва да бъде извлечена от паметта.
- **Стеков указател (SP/ESP):** Сочи към върха на системния стек в ОП.
- **Сегментни и контролни регистри:** Управляват режимите на работа на процесора и адресацията на паметта (CS, DS, SS).

3.2 Аритметико-логическо устройство (АЛУ)

АЛУ е комбинационна логическа схема, която извършва същинската обработка на данните. То приема операнди от вътрешните регистри и изпълнява над тях аритметични (събиране, изваждане, умножение) и логически операции (AND, OR, NOT, XOR, битови измествания).

3.3 Регистър на състоянието и флагове (Flags Register)

Това е специализиран регистър, в който всеки отделен бит действа като самостоятелен двупозиционен превключвател (**флаг**). Флаговете отразяват резултата от последното действие, извършено в АЛУ, и служат като вход за условните преходи. Основни флагове са:

- **ZF (Zero Flag):** Вдига се в 1, ако резултатът от операцията е нула.
- **CF (Carry Flag):** Указва пренос или заем от най-старшия бит при операции без знак.
- **OF (Overflow Flag):** Указва аритметично препълване при операции с допълнителен код (числа със знак).
- **SF (Sign Flag):** Равен е на най-старшия бит на резултата (индикира отрицателно число).

3.4 Блок за управление (Управляващо устройство - УУ)

Блокът за управление е "мозъкът" на процесора. Той извлича инструкциите, дешифрира ги и генерира поредица от вътрешни микрокоманди (управляващи импулси), които синхронизират работата на АЛУ, регистрите и шинните интерфейси в съответствие с тактовия генератор.

4 Инструкции на централен процесор

Машинната инструкция е завършена команда за процесора, представена в двоичен вид.

4.1 Структура на машинната инструкция

Една типична инструкция (например в x86 архитектурата) се състои от следните логически полета:

- **Префикси:** Незадължителни байтове, поставени преди кода на операцията, които модифицират поведението ѝ (например префикси за повторение при низове, пренасочване на сегменти или промяна на размера на операнда).
- **Код на операцията (Opcode):** Задължително поле, което идентифицира конкретната команда (например събиране, преход, преместване).
- **Спецификатори на операндите:** Полета, указващи местоположението на входните и изходните данни (регистър, адрес в паметта или директна стойност).

4.2 Модели на адресация на операндите

Адресацията дефинира начина, по който се изчислява реалният (ефективният) адрес на данните в паметта:

- **Директна (непосредствена) адресация:** Операндът е част от самата инструкция (напр. `MOV EAX, 5` – числото 5 е записано директно в кода).
- **Регистърна адресация:** Данните се намират в посочен регистър (`MOV EAX, EBX`).
- **Пряка (абсолютна) адресация:** Инструкцията съдържа точния числен адрес от ОП (`MOV EAX, [1000H]`).
- **Косвена регистърна адресация:** Регистър съдържа адреса към клетката от паметта (`MOV EAX, [EBX]`).
- **Базова индексирана адресация с отместване:** Адресът се изчислява динамично като сума от базов регистър, индексен регистър (евентуално мащабиран) и константно отместване (напр. при работа с масиви: `MOV EAX, [EBX + ESI*4 + 10H]`).

4.3 Функционални групи инструкции

4.3.1 Аритметико-логически инструкции

Извършват обработка в АЛУ. Примери в x86:

- **ADD / SUB:** Събиране и изваждане на операнди.
- **MUL / IMUL:** Умножение без знак и със знак (изискват детайлно внимание към разрядността и знаковия бит).
- **AND / OR / XOR / NOT:** Побитови логически операции.
- **SHL / SHR / SAR:** Логически и аритметични измествания на битове наляво-/надясно.

4.3.2 Низови инструкции (String Instructions)

Специализирани команди за обработка на последователности от данни в паметта (байтове, думи). Те автоматично инкрементират или декрементират индексните регистри (ESI и EDI) в зависимост от флага за посока (DF):

- **MOVS:** Копира елемент от низа, сочен от ESI, на адреса в EDI.
- **CMPS:** Сравнява елементи от два низа.
- **SCAS:** Сравнява елемент от низ със съдържанието на акумулатора (AL/AX/EAX).
- **LODS / STOS:** Зарежда в / записва от акумулатора символ в низа.

Тези инструкции често се комбинират с префикси за повторение (**REP, REPE, REPNE**), които зациклят изпълнението им, докато броячът ECX стане равен на 0 или се наруши условието за равенство.

4.3.3 Безусловни и условни преходи

Променят стандартния последователен поток на програмата чрез модифициране на програмния брояч (*IP/EIP*):

- **Безусловен преход (JMP):** Винаги прехвърля изпълнението към посочения адрес.
- **Условни преходи (JE, JNE, JG, JL и др.):** Изпълняват се само ако е изпълнено конкретно логическо условие, проверено чрез текущото състояние на флаговия регистър (напр. JE скача, ако $ZF = 1$).

4.3.4 Управление на програмата

Инструкции, които организират жизнения цикъл на програмната структура, функциите и прекъсванията:

- **CALL**: Извиква подпрограма (функция). Преди прехода записва текущия адрес за връщане в системния стек.
- **RET**: Връща изпълнението от подпрограмата, като извлича адреса за връщане от стека.
- **INT / INTO**: Предизвиква софтуерно прекъсване (преход към системна подпрограма на ОС).
- **HLT**: Спира изпълнението на инструкции от процесора и го привежда в чакащо състояние до следващ външен хардуерен импулс (прекъсване).